Quantum Load Balancing for HPC Clusters: QAOA Simulated at Scale with GPUs

Xu Xia Denzel Elden Wijaya Keerthivasan Efe Kahraman

SC4055 Group Project, November 2026

1 Introduction

High-performance computing (HPC) clusters run many jobs at once, and a scheduler decides which job runs on which compute node. A poor placement leaves one node overloaded while another sits idle. Placement must also respect interactions between jobs: two memory-bound jobs on the same node slow each other down, so schedulers let users declare *anti-affinity* rules that keep such jobs apart. Even splitting n jobs between two nodes has 2^n possible placements and is NP-hard in general.

The Quantum Approximate Optimization Algorithm (QAOA) [1] is a leading candidate for such problems on near-term quantum devices [2]. It rewrites the problem as an Ising Hamiltonian whose lowest-energy state is the best solution [3], prepares a low-energy state with a short parameterised circuit, and tunes the circuit's parameters with a classical optimiser. Large enough quantum hardware is not available to us, so we simulate QAOA. Simulation is itself an HPC problem: n qubits need 2^n complex amplitudes, so each extra qubit doubles the memory and the work. We use GPUs through NVIDIA's CUDA-Q platform [4], so HPC appears twice: as the application we schedule for, and as the tool that makes the study possible.

We place n jobs on two nodes so that the workloads are as balanced as possible while conflicting jobs stay apart (Fig. 1), and ask four questions.

RQ1 Quality. How close does QAOA get to the optimal schedule, and does it improve with circuit depth?

RQ2 Classical comparison. How does QAOA compare with simulated annealing, an exact solver and standard partitioning heuristics?

RQ3 Noise. How much do hardware errors degrade QAOA?

RQ4 HPC scalability. How much faster are GPUs than CPUs at simulating these circuits, and how large a problem can we reach?

We do not aim to show a quantum advantage: at sizes that can be simulated, classical methods win easily, and we show this directly. Instead we aim for a correct formulation, a reproducible implementation and an honest analysis of where QAOA works and where it struggles.

Section 9 summarises our answers to the four questions.

2 Background

Ising models. Many combinatorial problems can be written as an Ising model: minimise $C(z) = c_0 + \sum_i h_i z_i + \sum_{i<j} J_{ij} z_i z_j$ over spins $z_i \in \{+1, -1\}$ [3]. Constraints are added as penalty terms. Replacing each z_i by the Pauli operator Z_i gives a Hamiltonian H_C that is diagonal in the computational basis, whose eigenvalue on $|x\rangle$ is the cost of that assignment ($z_i = 1 - 2x_i$), so its ground state is the optimal solution.

QAOA. QAOA [1] starts from the uniform superposition $|+\rangle^{\otimes n}$ and applies p layers, each a cost layer $e^{-i\gamma H_C}$ followed by a mixer $e^{-i\beta H_M}$ with $H_M = \sum_i X_i$:

$$|\boldsymbol{\gamma}, \boldsymbol{\beta}\rangle = e^{-i\beta_p H_M} e^{-i\gamma_p H_C} \cdots e^{-i\beta_1 H_M} e^{-i\gamma_1 H_C} |+\rangle^{\otimes n}. \tag{1}$$

The cost layer gives each assignment a cost-dependent phase, and the mixer turns phase differences into amplitude differences, so with good angles low-cost assignments become more likely. A classical optimiser tunes the $2p$ angles to minimise $\langle H_C \rangle$, and the final circuit is measured many times, each shot giving one candidate solution. As $p \to \infty$ QAOA can imitate adiabatic evolution and reach the ground state [1]; at small p it is approximate, and because the energy landscape has many local minima, the starting angles matter. INTERP [5] starts depth $p+1$ by interpolating the optimised angles of depth p.

GPU simulation. A statevector simulator stores all 2^n amplitudes (8 GiB for 30 qubits in single precision, 16 GiB in double), and every gate updates every amplitude: many simple, independent updates to one large array, which suits GPUs. CUDA-Q [4] runs the same circuit on CPU or GPU simulators; noise needs a density matrix with 4^n entries, which limits our noise study to small n.

3 Problem Formulation

Scheduling model. We place n jobs on two identical nodes, A and B, with $z_i = +1$ for node A and -1 for node B. Job i has a normalised runtime $w_i = \tilde{w}_i / \max_k \tilde{w}_k \in (0, 1]$, and a conflict graph E lists pairs of jobs that should not share a node. With total work W and load difference $D = \sum_i w_i z_i$, the makespan is $(W + |D|)/2$, so minimising it is the same as minimising D^2. Without conflicts this is number partitioning [3, 6].

Instances. Runtimes are heavy-tailed, like real HPC jobs: $\tilde{w}_i = \max(1, \mathrm{round}(e^{X_i}))$ with $X_i \sim \mathcal{N}(2, 1)$. All conflicts can be satisfied only if E is bipartite, so we *plant* a hidden random split of the jobs into two halves and keep each pair across the split as a conflict with probability 0.2, so every instance has at least one conflict-free schedule. The runtimes are independent of the hidden split, so the solver must still trade balance against conflicts.

Cost function and Hamiltonian. The cost adds a penalty for conflicts to the balance term:

$$C(z) = \Big(\sum_i w_i z_i \Big)^2 + \lambda \sum_{(i,j) \in E} z_i z_j = \sum_i w_i^2 + \sum_{i<j} J_{ij} z_i z_j, \qquad J_{ij} = 2 w_i w_j + \lambda \mathbb{1}[(i,j) \in E]. \tag{2}$$

For a conflict edge, $z_i z_j = +1$ if both jobs share a node, so each violated conflict adds 2λ to the cost. The balance term couples every pair of jobs, so the Ising graph is complete with $n(n-1)/2$ couplings. Replacing z_i by Z_i gives the cost Hamiltonian

$$H_C = \frac{1}{s} \Big(\sum_i w_i^2 \, I + \sum_{i<j} J_{ij} \, Z_i Z_j \Big), \tag{3}$$

whose eigenvalue on $|x\rangle$ is $C(z)/s$.

Normalisation. We divide by $s = \max_{i<j} |J_{ij}|$ so that the largest coupling is exactly 1. This does not change the optimum, but gives every instance the same scale for γ, so that the energy does not oscillate too fast in γ for the optimiser and angles can be transferred between instances. Energies are converted back with $C = s \langle H_C \rangle$ before any metric is computed.

Mirror symmetry. Swapping the two nodes, $z \mapsto -z$, never changes the cost, so every schedule has a mirror image with the same cost, and QAOA gives both exactly the same probability. Integer runtimes often allow several balanced splits, so many instances have more than two optimal bitstrings, and $P(\text{opt})$ counts *all* of them.

Penalty weight. A small λ lets the optimum accept a conflict for better balance; a large λ dominates s and shrinks the balance couplings after scaling, which makes balance harder for QAOA to resolve. We swept $\lambda \in \{0.1, 0.5, 1, 2, 5\}$ on 20 instances at each even n from 6 to 24 and chose the smallest λ whose optimum has zero violations on at least 90 % of instances at every size. The value $\lambda = 0.1$ fails this test at most sizes and $\lambda = 0.5$ passes at every size, so we use $\lambda = 0.5$. We report violations as a separate metric for every method.

4 Method

Circuit. Each job is one qubit (Fig. 1). The circuit applies Hadamard gates and then p layers. Since all terms of H_C commute, the cost layer factorises exactly into $e^{-i\gamma J_{ij} Z_i Z_j} = \mathrm{CX}_{ij}\, R_Z^{(j)}(2\gamma J_{ij})\, \mathrm{CX}_{ij}$ for each coupling, and the mixer is $R_X(2\beta)$ on every qubit, so a layer has $n(n-1)$ CX gates. Since H_C is real, the angles $\boldsymbol{\theta} = (\boldsymbol{\gamma}, \boldsymbol{\beta})$ and $-\boldsymbol{\theta}$ give the same probabilities, so we flip every angle vector to $\gamma_1 \geq 0$ before comparing or averaging angles.

Optimisation. COBYLA (initial step 0.2 rad, at most 500 evaluations) minimises the exact energy $\langle H_C \rangle$, free of shot noise.

Initial angles. E1 starts $p = 1$ from a linear ramp and then uses INTERP [5], so each instance is optimised as a chain $p = 1, \ldots, 6$. E2 also tests the best of five random starts, and *transfer*: the median optimised angles of the ten $n = 8$ instances, used as a starting point or with no optimisation at all.

Metrics. After optimisation we draw 10,000 shots. The exact $C_{\min}$, $C_{\max}$ and optimal bitstrings come from brute force for $n \leq 24$ (CP-SAT above). We report the approximation ratio of the mean cost, $r = (C_{\max} - \langle C \rangle)/(C_{\max} - C_{\min})$, for which random guessing already scores 0.71–0.73; the success probability $P(\text{opt})$, the fraction of shots that hit any optimal bitstring; and the best of the 10,000 shots, which is what QAOA would return in practice and the fair comparison with classical solvers.

Validation. 40 automated tests check, among others, that sampled and exact energies agree, that every bitstring and its mirror are equally likely (Section 3), and that optimised QAOA beats random sampling.

5 Implementation

The QAOA circuit is one CUDA-Q kernel that takes the couplings and angles as inputs, so the same code serves every instance and depth. During optimisation, `observe` returns the exact energy without shots; once the angles are fixed, `sample` measures the final circuit. We use the CPU target `qpp-cpu` (always fp64), the GPU target `nvidia` (fp32 by default, fp64 with one option) and `density-matrix-cpu` for noise. Each run is saved to a CSV row with its seed, target and precision as soon as it ends, so an interrupted job restarts where it left off.

6 Experimental Setup

All GPU runs and CPU timing runs ran as Slurm batch jobs on a shared university cluster, whose nodes each have two 48 GB NVIDIA A40 GPUs, two 16-core CPUs and 192 GB of memory (Python 3.11, CUDA-Q 0.16, SciPy 1.17). Each GPU job used one GPU with a 24-hour limit, and CPU timing runs used 8 threads. Table 1 lists each experiment, all with $\lambda = 0.5$. Because E1 mixes CPU fp64 and GPU fp32, we also ran $n = 12$ on the GPU: the ratio changed by less than 0.001 in 58 of 60 runs and the best sampled cost never changed.

Table 1: Settings of each experiment. Instances are counted per size; sizes go up in steps of 2 except in E3.

Exp.	Purpose	Simulator	Sizes n	Instances	Depth p
E1	Depth sweep	CPU fp64 / GPU fp32	8–12 / 14–20	10	1–6
E2	Starting angles	GPU fp32	10–16	10	2, 4, 6
E3	Classical methods	none	8–20 / 21–28	10 / 1	none
E4	Timing	CPU fp64 / GPU fp32, fp64	10–20 / 10–30	3	3
E5	Reused angles	GPU fp32	20–28	10	1–6
E6	Gate noise	CPU density matrix	6, 8	10	1–4

7 Results

Unless stated otherwise, every point is a mean over the same ten instances per size (seeds 0–9), and shaded bands show ± 1 standard error.

7.1 E1: Solution quality and circuit depth (RQ1)

E1 runs INTERP-initialised QAOA at $p = 1, \ldots, 6$ on ten instances per size $n = 8, 10, \ldots, 20$.

Quality improves with every layer (Fig. 2, top): from about 0.72 for random guessing to 0.93–0.97 at $p = 6$. No instance ever got worse with depth, but each extra layer adds less. $n = 14$ and 16 end below $n = 18$ and 20 because of two instances whose INTERP chain starts in a poor basin and never recovers; without them both sizes average 0.95.

$P(\text{opt})$ at $p = 6$ falls from 0.24 at $n = 8$ to 0.035 at $n = 20$ (Fig. 2, bottom), but random guessing falls as 2^{-n}, so QAOA's advantage grows from 19 to about 17,000 times. From $p = 3$ the best of 10,000 shots is an exact optimum on all 70 instances.

7.2 E2: Initialisation strategies

E2 compares four starting points at $p = 6$ on the same ten instances per size $n = 10$–16: INTERP, the best of five random starts, and angles transferred from the $n = 8$ instances of E1, with and without optimisation.

Transfer followed by optimisation ties INTERP at $n = 10$ and gives the best quality at every larger size, with a mean ratio of 0.954 against 0.898 for random starts (Fig. 3). It is also cheap: about 360 evaluations, against 910 for INTERP's full chain and 2,430 for random starts. Its lead over INTERP comes from only two instances, the two on which INTERP is stuck in E1; without them the two methods are level. Even with no optimisation, transferred angles reach 0.933, so good angles are similar across instances and sizes, as reported for other problems [7, 8], which E5 exploits.

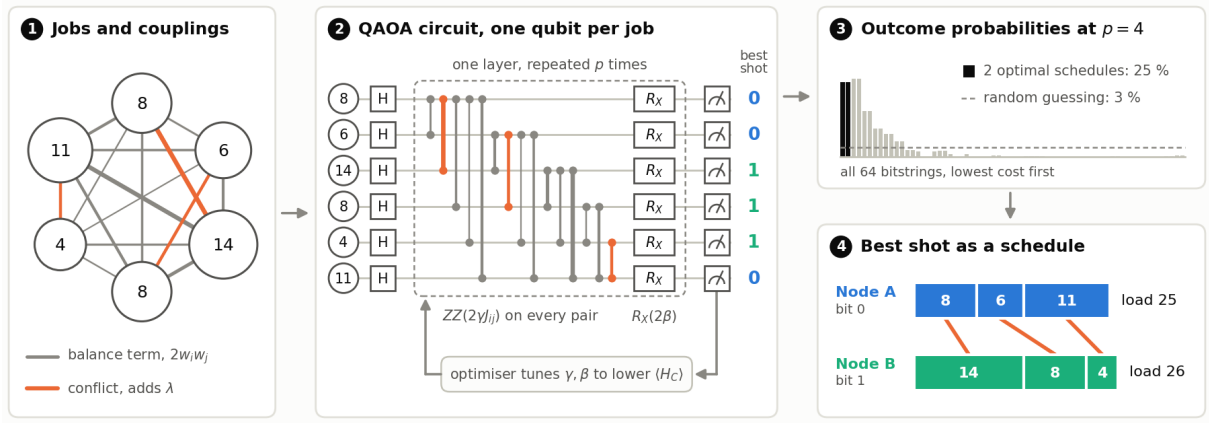


Figure 1: The method on one real six-job instance. (1) Jobs with their runtimes; line width is coupling strength and orange marks a conflict. (2) One ZZ gate per coupling. (3) Exact probabilities with the angles from E6. (4) An optimal bitstring read as a schedule.

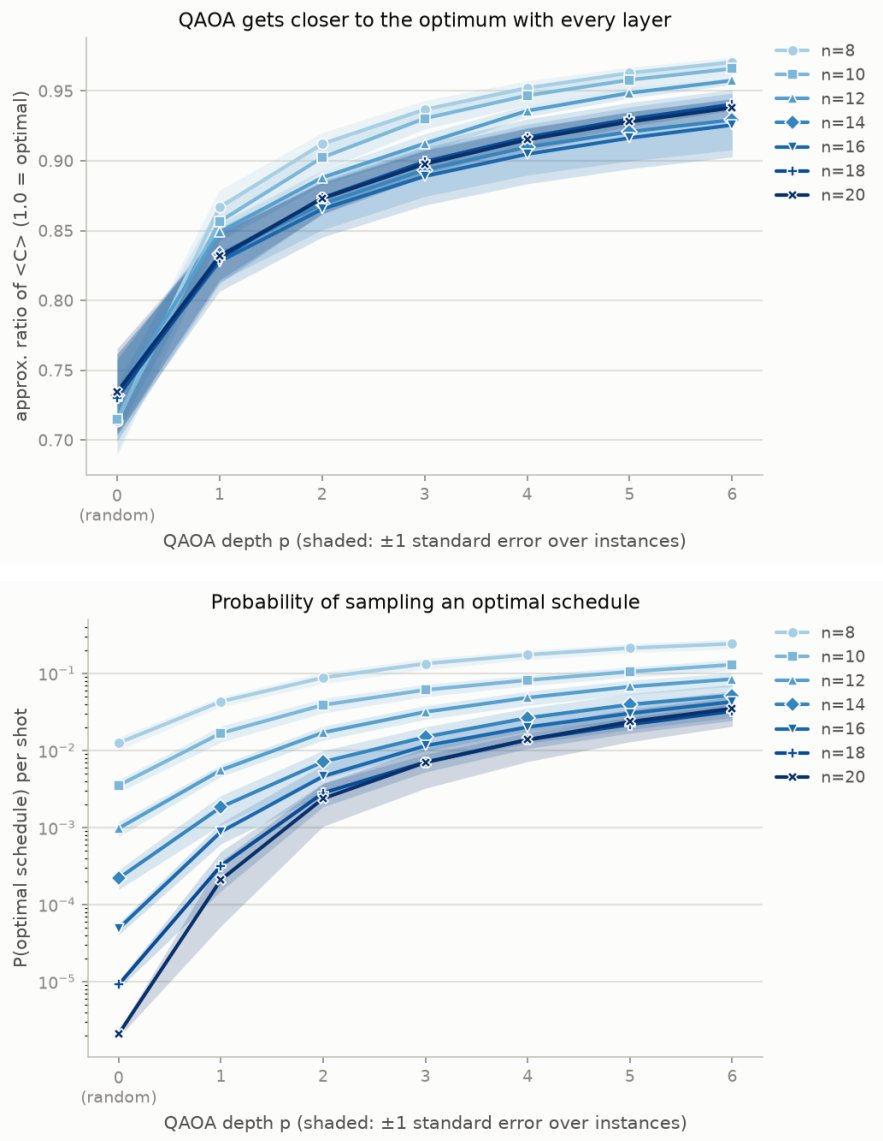


Figure 2: E1. Top: approximation ratio of the mean cost against depth. Bottom: probability that one shot is optimal (log scale). $p = 0$ is random guessing.

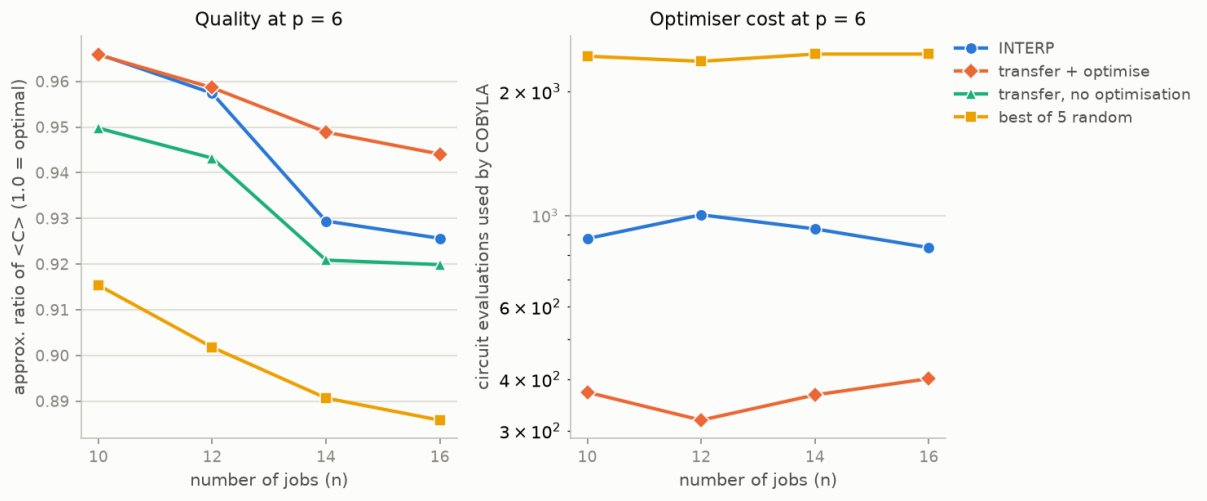


Figure 3: E2 at $p = 6$. Left: approximation ratio of the mean cost. Right: circuit evaluations used by COBYLA (log scale); for INTERP this counts the whole chain $p = 1, \ldots, 6$.

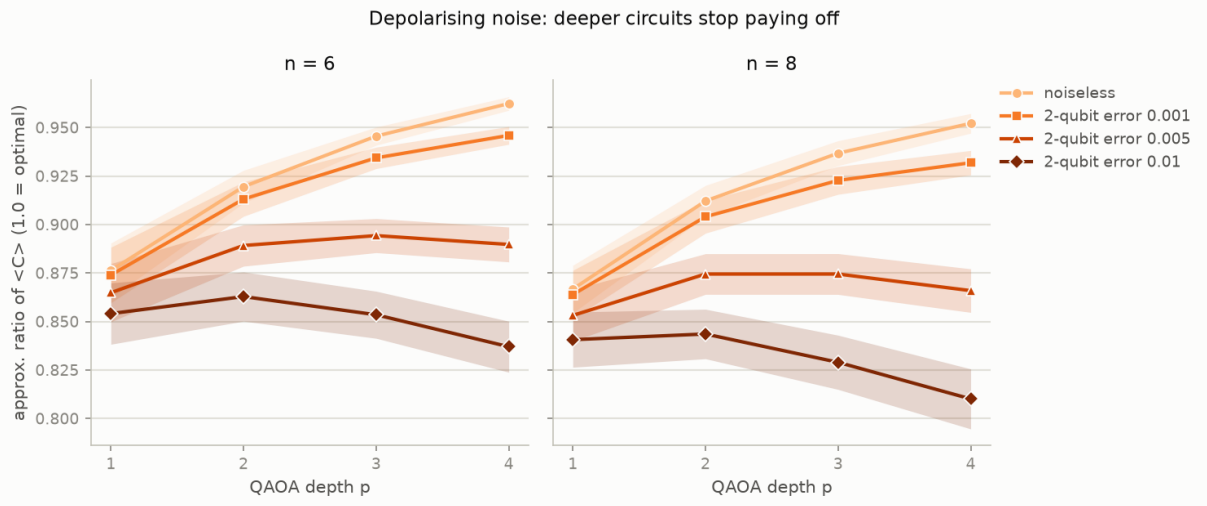


Figure 4: E6. Approximation ratio of the mean cost against depth for each two-qubit error rate, at $n = 6$ (left) and $n = 8$ (right).

7.3 E3: Classical baselines (RQ2)

E3 runs four classical methods: Karmarkar–Karp (KK) and greedy LPT, which balance load only and ignore conflicts, and simulated annealing (SA) and CP-SAT, which optimise the full cost. Each size $n = 8$–20 has ten instances (E1's at even n); $n = 21$–28 have one each.

KK and LPT fall from about 0.85 at $n = 8$ to about 0.77 at $n = 20$, because larger instances have more conflicts to violate: their mean violations per solution grow from about 1 to about 9. SA and CP-SAT are effectively exact: CP-SAT is an exact solver, SA reaches the optimum in 137 of 138 runs, and both leave almost no violations. KK and LPT take microseconds, CP-SAT 3.9 ms on average and SA 126 ms.

QAOA's mean-cost ratio of 0.93–0.97 beats KK and LPT at every size (Fig. 5). Against SA and CP-SAT the fair comparison is QAOA's best shot, the schedule one would actually use, and from $p = 3$ it is optimal on every instance, so QAOA matches them on quality. It loses on cost: one QAOA evaluation at $n = 20$ already takes 31 ms on a GPU and an optimisation needs hundreds, while CP-SAT solves the whole problem exactly in milliseconds.

7.4 E4: Backend scaling (RQ4)

E4 fixes the circuit at $p = 3$ and changes only the target. Runs stop after different numbers of evaluations, so we compare time per evaluation. Two settings hit the 24-hour limit, leaving one of three instances for GPU fp64 at $n = 30$ and two of three for the CPU at $n = 20$.

At $n = 20$ the CPU (fp64) takes 76 s and the GPU 31 ms in fp32, about 2,500 times faster, or 1,300 times at equal precision (GPU fp64: 60 ms; Fig. 6, left). Each extra qubit doubles the amplitudes and the gate count also grows with n, so the CPU's cost grows five- to six-fold every two qubits, while the GPU stays nearly flat until about $n = 22$, where the work starts to fill it.

fp32 and fp64 cost about the same on the GPU up to $n = 18$, but fp64 is about six times slower from $n = 24$. That is more than the $2\times$ of memory traffic alone, as the A40 has much lower fp64 throughput. Measured GPU memory matches 2^n amplitudes at 8 or 16 bytes each from $n = 24$ (Fig. 6, right), so one 48 GB A40 holds at most 32 qubits in fp32 and 31 in fp64.

7.5 E5: Large problems with transferred angles (RQ4)

Optimising fresh angles at $n = 20$–28 is too slow, so E5 takes the median angles E1 optimised at $n = 20$ and samples with them at every size up to $n = 28$, with no optimisation (ten instances per size, 10,000 shots, GPU fp32).

Quality degrades gently: at $p = 6$ the ratio falls from 0.919 at $n = 20$ to 0.897 at $n = 28$ (Fig. 7, left), although the search space grows 256-fold, and depth still helps throughout. At $n = 20$, reusing angles instead of optimising each instance costs 0.02 in ratio and halves $P(\text{opt})$. $P(\text{opt})$ at $p = 6$ falls to 0.008 at $n = 28$ (Fig. 7, right), where a run takes about 12 s.

7.6 E6: Robustness to noise (RQ3)

E6 takes the noiselessly optimised angles for ten instances at $n = 6$ and 8 and $p = 1$–4, and runs them on CUDA-Q's density-matrix simulator under depolarising noise: each CX gate fails with probability $p_{\text{err}} \in \{0, 0.001, 0.005, 0.01\}$ and each single-qubit gate with $p_{\text{err}}/10$.

At $p_{\text{err}} = 0.001$ quality still improves with every layer and ends only 0.02 below the noiseless value at $n = 8$ and $p = 4$ (Fig. 4). At 0.005 it stops improving after two or three layers. At 0.01 depth becomes harmful: at $n = 8$ the ratio falls from 0.841 at $p = 1$ to 0.810 at $p = 4$, and all ten instances end lower. Each layer adds two CX gates per pair of jobs, 56 at $n = 8$, so at $p = 4$ only $0.99^{224} \approx 10\,\%$ of runs avoid every CX error. So the best depth depends on the error rate.

8 Discussion

Strengths and weaknesses of QAOA. Good angles are similar across instances and sizes, so transferred angles beat INTERP at lower cost (E2) and carry QAOA to $n = 28$ with no optimisation (E5), which matters most on real hardware, where every evaluation is slow and noisy. QAOA's main cost is the optimisation loop: hundreds of evaluations, against milliseconds for CP-SAT (E3). $P(\text{opt})$ also falls with n, so larger problems need more shots, and noise caps the useful depth at $p = 1$–2 at a 1 % error rate (E6).

GPU and precision. Our 2,500-fold GPU speed-up at $n = 20$ is worth only about 11 extra qubits, because each qubit doubles the cost, and one 48 GB A40 holds at most 32 qubits in fp32. Single precision was the right choice on the A40. From 24 qubits on it ran about six times faster than fp64, and in our check at $n = 12$ it never changed the best cost found.

Limitations. We schedule onto only two nodes; k nodes would need several qubits per job and constraints to keep each job on exactly one node. Above $n = 24$ we know only one optimum and its mirror, so $P(\text{opt})$ there is a lower bound. Our noise model is depolarising only: it leaves out readout errors, decoherence and the SWAP gates that limited qubit connectivity would add, and we do not re-optimise angles under noise. Our timings are for simulation, not hardware.

9 Conclusion

We studied QAOA for two-node load balancing with anti-affinity constraints, simulated on GPUs with CUDA-Q. QAOA improves with every layer, from 0.72 for random guessing to 0.93–0.97 at $p = 6$, and its best sample is optimal on every instance from $p = 3$ (RQ1). It matches simulated annealing and CP-SAT on its best sample and beats greedy heuristics, but classical solvers are far faster at these sizes (RQ2). At a two-qubit error rate of 1 %, depth beyond $p = 2$ makes results worse (RQ3). An A40 GPU simulates these circuits about 2,500 times faster than a CPU at $n = 20$ and reaches 30 qubits, and transferred angles carry QAOA to 28 jobs without optimisation (RQ4). Next steps are scheduling onto more nodes, multi-GPU simulation beyond 30 qubits, and transferred angles on real hardware.

References

[1] Edward Farhi, Jeffrey Goldstone, and Sam Gutmann. A quantum approximate optimization algorithm. *arXiv preprint arXiv:1411.4028*, 2014.

[2] John Preskill. Quantum computing in the NISQ era and beyond. *Quantum*, 2:79, 2018.

[3] Andrew Lucas. Ising formulations of many NP problems. *Frontiers in Physics*, 2:5, 2014.

[4] NVIDIA. CUDA-Q: a platform for hybrid quantum-classical computing. Software, 2025.

[5] Leo Zhou, Sheng-Tao Wang, Soonwon Choi, Hannes Pichler, and Mikhail D. Lukin. Quantum approximate optimization algorithm: Performance, mechanism, and implementation on near-term devices. *Physical Review X*, 10:021067, 2020.

[6] Narendra Karmarkar and Richard M. Karp. The differencing method of set partitioning. Technical report, University of California, Berkeley, 1982.

[7] Fernando G. S. L. Brandão, Michael Broughton, Edward Farhi, Sam Gutmann, and Hartmut Neven. For fixed control parameters the quantum approximate optimization algorithm's objective function value concentrates for typical instances. *arXiv preprint arXiv:1812.04170*, 2018.

[8] Alexey Galda, Xiaoyuan Liu, Danylo Lykov, Yuri Alexeev, and Ilya Safro. Transferability of optimal QAOA parameters between random graphs. In *IEEE Int. Conf. on Quantum Computing and Engineering (QCE)*, pages 171–180, 2021.

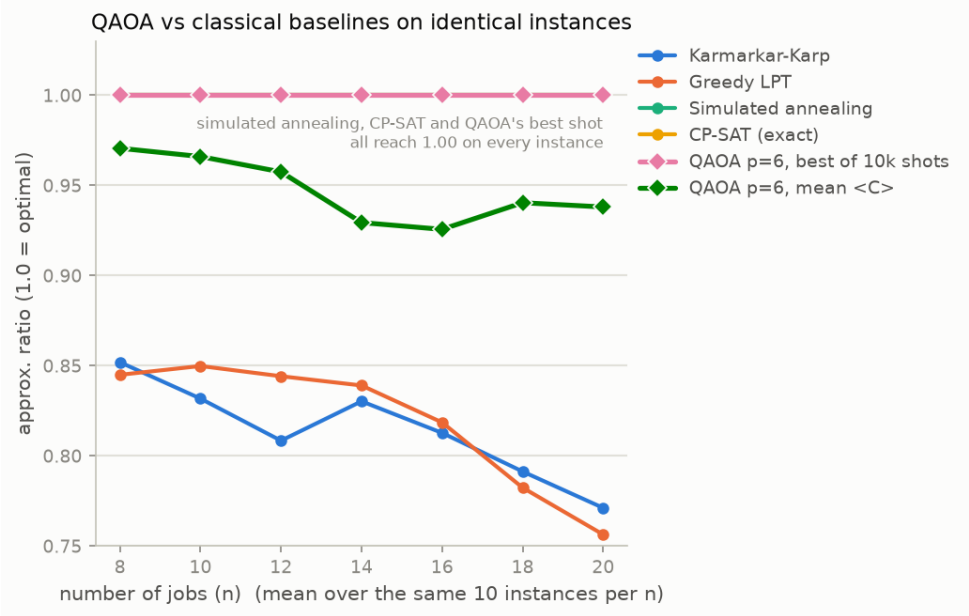


Figure 5: E3. QAOA at $p = 6$ (mean cost and best of 10,000 shots) against the four classical baselines at $n = 8$–20.

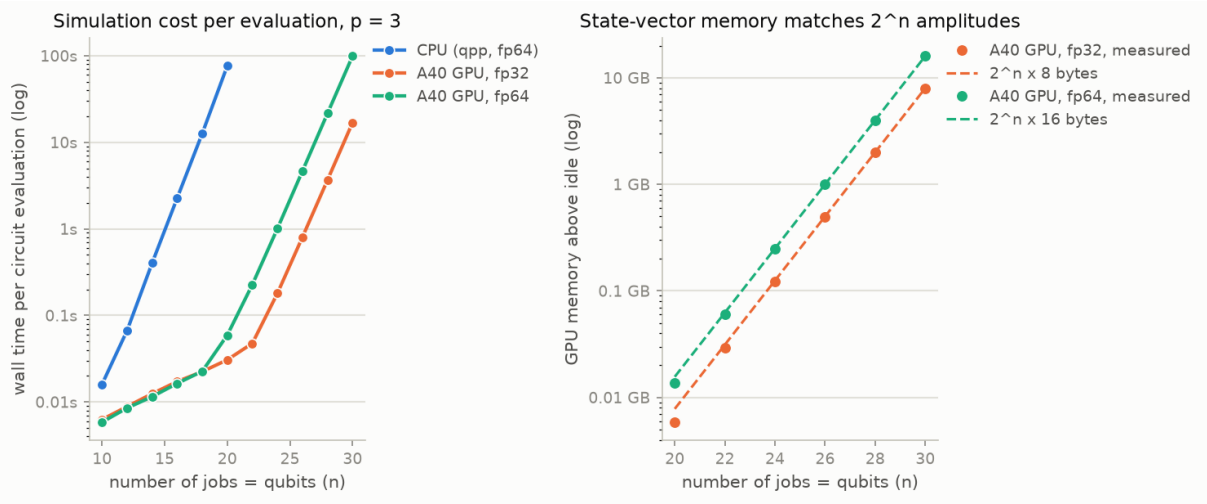


Figure 6: E4 at $p = 3$. Left: wall time per circuit evaluation (log scale). Right: GPU memory above an idle CUDA context against the 2^n-amplitude prediction.

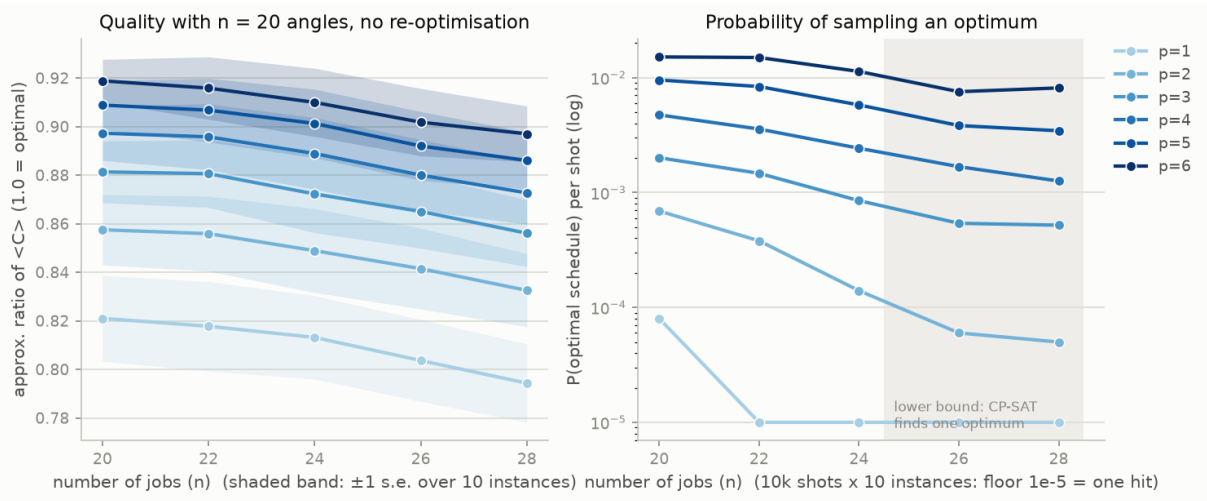


Figure 7: E5. Angles optimised at $n = 20$, reused without optimisation at $n = 20$–28. Left: approximation ratio by depth. Right: $P(\text{opt})$ (log scale), a lower bound above $n = 24$.